Final System Report

College0 — Web-based graduate program management

Team Y
Atai Kydyrov · Syed Mahmood · Abdullah Zidan · Justin Jones

Due: 11:59 pm on demo day — **May 14** (per course instructions)

1. Introduction

College0 is a full-stack academic operations system: visitors explore public statistics and apply for admission; registrars advance semester phases, define classes, resolve applications and complaints; students enroll under capacity and conflict rules, write reviews, use AI assistance scoped to their enrollments; instructors manage waitlists, grade, and file complaints. The implementation centers on **Supabase** (PostgreSQL with row-level security, Auth, Edge Functions, `pgvector`) plus a **React 19** single-page app built with **TypeScript**, **Vite**, **Tailwind CSS v4**, and **React Router v7**. Optional hosting uses **Vercel** for the static frontend; database and server logic remain on Supabase.

Install & run from scratch

1. Clone the repository (see §4). Open `college0-app/`.
2. `npm install` then copy `.env.example` → `.env` and set `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`.
3. Create a Supabase project; enable **uuid-ossp** and **vector**. Apply SQL migrations in `college0-app/supabase/migrations/` in filename order (see root `README.md`).
4. Add `SUPABASE_SERVICE_ROLE_KEY` to `.env`; run `npm run seed:registrar` for the first registrar account (or create a user in the Dashboard with metadata `role: registrar`).
5. Deploy Edge Functions from `college0-app/` with the Supabase CLI; set secrets `GEMINI_API_KEY`, etc., per README.
6. `npm run dev` — SPA at localhost. Production: connect the GitHub repo to Vercel with root `college0-app`.

2. Spec correspondence (1–10)

Relative emphasis of implemented areas (illustrative)

#	Required feature (summary)	Status	How it is implemented (concise)
1	Public GUI: intro, highest/lowest rated classes, top GPA students.	Finished	`PublicDashboard` loads view `v_public_dashboard_stats` (top/bottom classes by `avg_rating`, top GPA students). Home copy introduces the program. Anonymous display for leaderboard where appropriate.
2	Visitor student/instructor applications; registrar accept/reject; auto-accept GPA & quota; justification on override; credentials for accepted students.	Finished	Visitor forms insert `applications`. Registrar UI + `rpc_decide_application` (reject) and Edge `accept-student-application` (accept, creates Auth user, `student_id`, temp password). Auto-accept uses **operational semester** quota and **prior GPA ≥ 3.0** (implementation uses inclusive 3.0 and phases registration|running|grading). Instructor accept via same Edge path. `FirstLoginPage` forces password change when `first_login`.
3	Four semester periods; setup classes/time/instructor/size; registration 2–4 courses; conflicts; capacity/waitlist; instructor promotes; retake after F.	Finished	`rpc_transition_semester_phase`; registrar class form + calendar fields; `rpc_enroll_in_class` enforces phase, capacity (waitlist), schedule overlap, retake-on-F; `rpc_promote_waitlist` / `rpc_reject_waitlist`.
4	Running: no registration except special window; <2 courses warning; <3 students cancel class; special registration; instructor warnings; all-cancelled suspend.	Finished	Triggers/RPCs: under-enrollment warnings; `rpc_course_cancellation_scan` (running phase only) cancels class, drops enrollments, flags `special_registration_eligible`, student warnings, instructor warning; suspension when all classes cancelled. Student enroll page respects special flag.
5	Reviews with stars; anonymous to instructor; avg <2 instructor warning; 3 warnings suspend; lock after grade; taboo 1–2 masked +1 warn; ≥3 hidden +2 warns.	Finished	Edge `submit-review` + `reviews` RLS; `recalc_class_avg_rating` excludes hidden; `classes_after_low_avg_rating_warn`; instructor-facing queries omit author; `rpc_post_grade` locks reviews; taboo counts from registrar list with masking and warnings per policy.

#	Required feature (summary)	Status	How it is implemented (concise)
6	Grading period grades; missing-grade warnings; class GPA band registrar scrutiny; student GPA termination rules; honor roll; graduation application.	Partial	Grades via `rpc_post_grade` + phase enforcement; post–grading-period missing-grade warnings; low/high class average instructor warnings; profile triggers for GPA <2, double-F, 2–2.25 interview warning; honors counters and `rpc_redeem_honor_for_warning`; graduation workflow with registrar decision. *Gap:* "Registrar questions / fire instructor" for GPA band is reflected as **warnings/flags in UI** rather than a dedicated formal hearing workflow.
7	Complaints student↔registrar, instructor↔registrar; registrar resolution; 3 warnings suspend + fine.	Finished	`complaints` + `rpc_resolve_complaint`; fines table and automation per migrations; suspension on warning thresholds.
8	Role dashboards: instructor roster/academic info; student own records + tutorial; registrar sees all.	Finished	Layouts under `/instructor`, `/student`, `/registrar`; RLS scopes data; first-login tutorial; instructor grading/complaints include roster student fields where policy allows.
9	AI with local vector store + LLM fallback; role-scoped context; hallucination notice.	Finished	Edge `ai-chat`: `match_documents` on `document_embeddings` (768-d Gemini), then Gemini generate; prompts include visitor/student/instructor context blocks; UI warns about hallucinations when using general model answers.
10	Creative enrichment.	Finished	**(a)** Public **course map** (`/class-locations`) with Leaflet/OSM pins from `classes.location_lat/lng/label`; registrar required pin+label on create + `rpc_set_class_location` for edits. **(b)** `study-groups` Edge function. **(c)** Directory / visitor-safe enrollment views where specified.

3. Team contributions & grading template

The table below records **equal baseline contribution (25% each)** for a four-person team, with no bonus or penalty applied in this submitted version. Per course policy, any non-equal split, bonus (max +10% total reallocated), or penalty must be agreed in writing by all members — use the signature blocks.

Member	Primary contributions (examples)	Suggested %	Bonus	Penalty
Atai Kydyrov	Supabase linkage, deployments (Vercel/Edge), environment & tooling, integration QA.	25%	—	—
Syed Mahmood	Backend migrations/RPC review flows, auth & application acceptance paths, data integrity checks.	25%	—	—
Abdullah Zidan	Registrar & student UX, enrollment and semester flows, spec-to-UI traceability.	25%	—	—
Justin Jones	Instructor features, grading and roster surfaces, complaints and reviews UX.	25%	—	—

Signatures (required if percentages differ from equal split or if bonus/penalty is applied):

Atai Kydyrov — date: _____ Syed Mahmood — date: _____ Abdullah Zidan — date: _____ Justin Jones — date: _____

4. Git repository

Clone URL (HTTPS):

```
https://github.com/CoderNameFaisal/College0.git
```

> **Visibility:** Ensure this repository is set to **Public** (or grant the instructor access) so it can be cloned for grading. Application source lives under `college0-app/` ; database migrations under `college0-app/supabase/migrations/` .

5. Database schema (core)

PostgreSQL schema `public` is defined incrementally by ordered migrations. Auth identities live in `auth.users`; `public.profiles` mirrors each user with `role`, academic fields, and status. Below is a logical ER sketch of principal entities and foreign keys.

```
auth.users (id)
    │ 1:1
    ▼
profiles (id PK/FK, role, full_name, student_id, status, cumulative_gpa, …)
    │
    ├──< semesters (id) ──< classes (id, semester_id, instructor_id→profiles,
    │                      course_code, title, schedule_time, max_students,
    │                      avg_rating, is_cancelled, location_lat/lng/label, …)
    │         │
    │         └──< enrollments (id, student_id→profiles, class_id→classes,
    │                           status, grade, semester_id, …)
    │
    ├──< applications (visitor/instructor pipeline)
    ├──< reviews (class_id, author_id, stars, body, filtered_body, is_hidden)
    ├──< warnings (target_id, reason, semester_id, …)
    ├──< complaints (filed_by, against, …)
    ├──< graduation_applications
    ├──< study_groups
    ├──< taboo_words
    ├──< fines
    ├──< document_embeddings (content, embedding vector(768), …)
    └──< audit_log, required_courses, student_semester_honors, …
```

Security model

Row Level Security (RLS) policies on tables such as `profiles`, `classes`, `enrollments`, `reviews`, and `complaints` enforce role-based access; sensitive operations (phase transitions, scans, grade posting, etc.) are additionally guarded by `SECURITY DEFINER` RPCs checking `_current_role()`.

6. Closing

College0 satisfies the majority of the written specification through a combination of Postgres constraints/triggers, explicit RPCs, Edge Functions for Auth-adjacent workflows and AI, and a role-organized React client. Items called out as **partial** above should be reviewed first in any future sprint.

Report generated as static HTML for Team Y — print to PDF for submission. Not part of the production bundle.